

9. How to Leverage the User Interface in Event-Driven Microservice Architectures

Hugo Filipe Oliveira Rocha¹

(1) Ermesinde, Portugal

This chapter covers:

- Using an aggregating layer with API composition to build information for the UI
- Applying UI composition both by modules, pages, and sections to interact with distributed data
- Using backends for frontends to provide a denormalized view of the data
- How task-based UIs can be relevant to manage the asynchronicity of event-driven architectures
- Using event-driven APIs to support responsive UIs

As we build a fully segregated, highly scalable, top-of-the-line microservice architecture platform, we tend to forget who's at the end, using the functionality, often a user interface. You might have heard of the three-mile island incident in Pennsylvania in 1979, where a nuclear power station's reactor partially melted and caused a radiation leak that led to the evacuation of thousands. It is the most significant incident of commercial nuclear power plant history of the United States. Even though there were no injuries, it had the potential to make the whole area uninhabitable for the foreseeable future. The entire chain of events that caused the incident originated in a misunderstanding in a UI. The nuclear reactor primary system's relief valve became stuck and stayed open for an extended period after a failing close command, allowing large quantities of nuclear reactor coolant to escape. The lack of coolant led to the re-

actor meltdown. Why didn't the operators close the valve manually? A light in the interface lit up when the valve was open and went out when the valve was closed. But the light went out when the system sent the signal to close the valve, not when the valve was physically closed. So, the computer sent the signal, the light went out, but the valve didn't actually close, which led to the operators not noticing the valve was open. If you work around event-driven services and eventual consistency for enough time, this type of issue feels too familiar. There is a difference between informing the user that an action was done and waiting to signal the change only after it took place.

Not all of us have the sole mission to build a microservice platform; some of us have the pristine purpose of delighting users with aweing interfaces. Although I would take the challenge of handling eventual consistency over aligning every element in a reactive web page every day, there is an eye-watering wonder in watching a complex architecture do its work through a UI perspective.

Microservices add a layer of complexity to traditional UI approaches; fetching information becomes a non-trivial task due to the high data fragmentation. Some use cases might need an aggregated view of data from different domains. When each domain is isolated by a segregated service with an independent database, retrieving an aggregated view of the data across domains can be exceptionally challenging. The platform exposed by the microservice architecture should be agnostic of the user interface since a myriad of clients can access it; there can be desktop users, mobile users, even outside interactions with APIs. The platform should support those use cases. However, being abstract enough for the different use cases often brings challenges on how to answer specific use cases with acceptable performance.

This chapter will work through some examples of how we can overcome these challenges and explore some approaches to implement communication with the user interfaces. Although some of them can be tackled by the UI layer, they might be inappropriate for more complex, data-intensive use cases. Event-driven architectures are built in a way where we can easily share, transform, and build different views of data accom-

modated for more complex use cases. Events are also aligned with the reactive nature of UIs; building a bridge between the two can be exceptionally valuable, as we will discuss at the end of the chapter.

9.1 Using an Aggregating Layer to Build a UI in a Distributed Microservice Architecture

In a typical monolithic application, the backend could feed the UI with aggregated information in a trivial way since all data often sits in the same database. As we segregate each domain and distribute the domain's data into separate services with independent databases, fetching information from different domains can be cumbersome. This section will approach the use of an aggregating layer to facilitate the fetch of data in the UI and discuss the impacts of API composition.

Let's say we have a UI for an eCommerce platform that displays the product listing, the user's current order information, and the recommendations for similar products that the user bought recently. Ideally, we want to retrieve all data from the backend in a single request and display it in the interface. However, different parts of the data will likely sit in different domains and different services. Figure [9-1](#) illustrates this example.

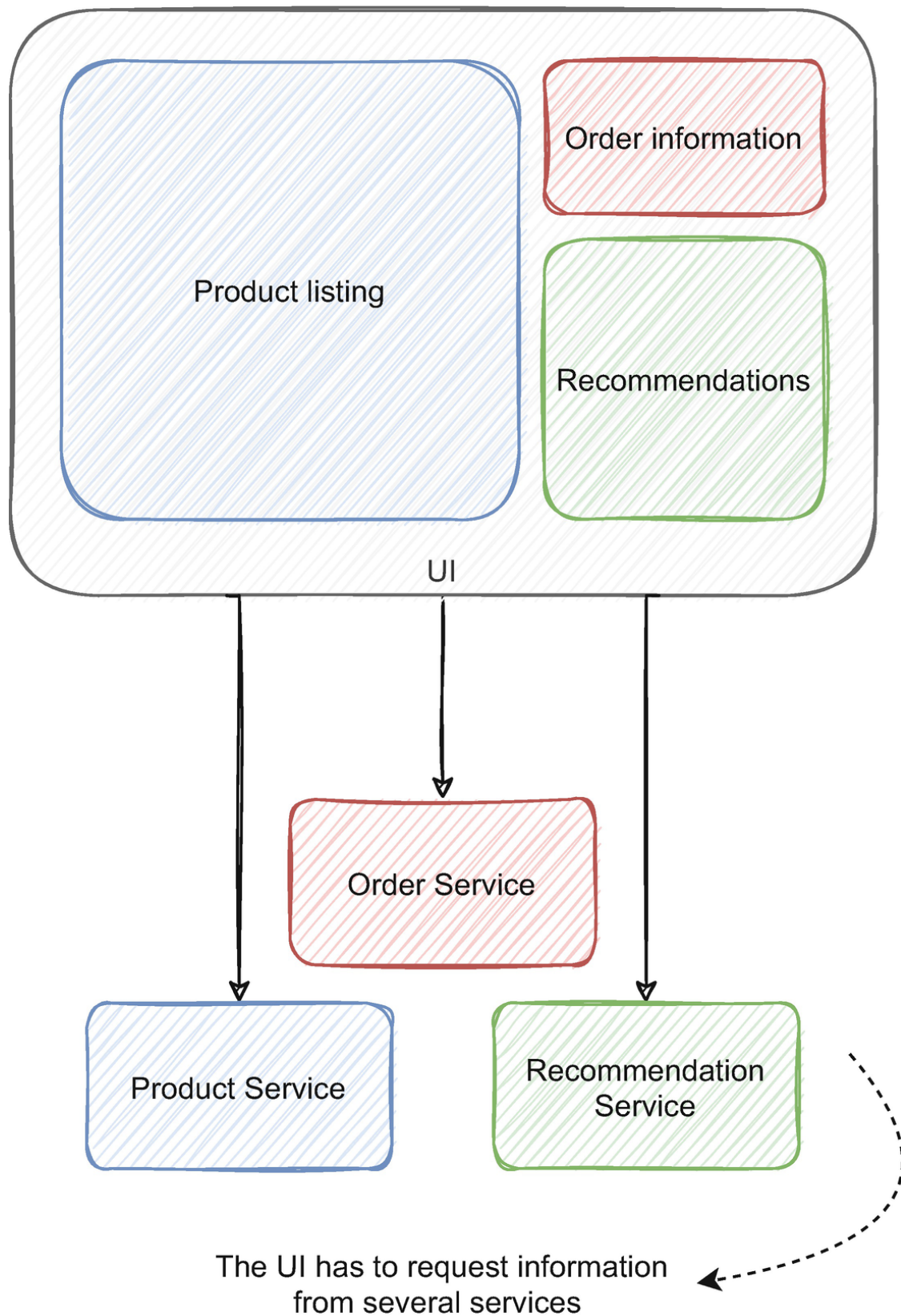


Figure 9-1 A UI fetching different information from several services

The filtering and aggregation logic would also be centralized in the single frontend application and in one deployable unit. This kind of approach usually suffers from the drawbacks of monoliths we discussed in Chapter 1. Releasing new functionality when several teams have ownership of the same application might be challenging to manage.

Also, the UI has to make three separate requests to fill a single screen. The excess of requests can be troublesome due to mobile users' network speed and bandwidth limitations, for example. The data fetched from the microservices might not be adapted to the needs of the UI, for example, the product service might return all the product information even though the UI might only need the product name.

A typical approach we can use to improve this situation is to use an aggregating layer between the microservices and the UI. Instead of the UI requesting information from each microservice and aggregating it, it would request the information from the aggregating layer, as illustrated in Figure [9-2](#).

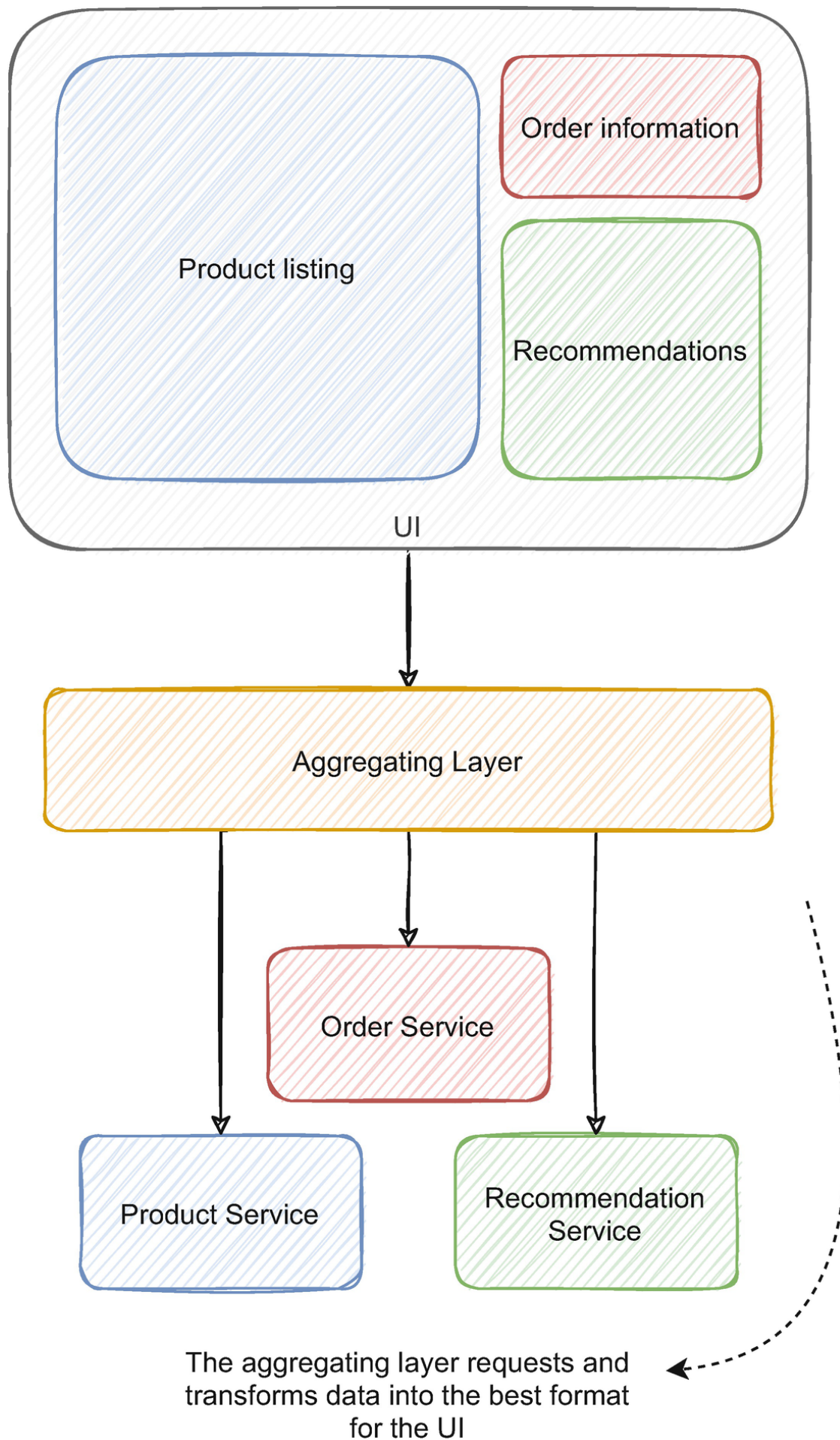


Figure 9-2 Using an aggregating layer to fetch and transform data for the UI

The aggregating layer has the logic to fetch, join, and transform data in the best format for the UI. This way, instead of making three separate requests, the UI makes one that returns only the required information. By decreasing the number of requests and the size of the data requested by the UI, we optimize the request's response time and the required bandwidth.

Although we optimized the requests for the UI, this generic component in charge of orchestrating the requests to the microservices might become a monolith in its own right. As we add more microservices and the different types of UIs keep growing, so does the aggregating layer. As the UI supports various use cases, user experiences, and devices, managing all the different requirements in a single application can be challenging. This kind of segregating layer between the internal microservice platform and the outside clients tends to grow and become a bottleneck to delivery.

These two patterns, a single frontend application and using an aggregation layer, are typically best for small applications with full-stack teams that develop both the UI and the platform. As with monoliths, it's often beneficial to resort to these patterns while the application is simple and doesn't support many requirements. As the application grows, the more challenging it is to manage.

9.2 Backends for Frontends (BFFs)

An aggregating layer can be challenging to manage as the application grows and incorporates more use cases and different user experiences. Chapter 1 discussed how a monolith could become a constricting knot to business growth; an aggregating layer can suffer from the same drawbacks. Since it is a piece that glues the frontend and backend together, it has a pivotal role in the continuous delivery of value. For example, if we need to add more product information to a product listing, we might not only need to change the product listing UI and the product service but also the aggregating layer. If every other UI needs to do the same, all deliveries touch the aggregating layer, potentially becoming a bottleneck. This section will discuss backends for frontends as a way to solve that challenge.

The functionalities we expose on our microservice platform are often designed to be generic and agnostic of the use case. For example, our platform might expose several REST endpoints with conceptual resources for the external applications to interact with, like order, product, stock, etc. In fact, the functionalities supported by the microservices are focused on their domain and usually exposed as a generic concept any interface can interact with. For example, we might design the order service in light of the order fulfillment business flow and expose it according to that design, which might or might not be aligned with the needs of the UIs. This mismatch is not necessarily a bad thing; the microservice platform should follow the domain concepts and expose more conceptual functionalities since it can be used by a myriad of use cases, ranging from graphical user interfaces to integrations with external systems. However, the mismatch between the platform functionalities and the needs of the UIs needs to be addressed somewhere. As we discussed, an aggregating layer could be an option; however, it can quickly become bloated and struggle to keep up with each different use case.

That's where backends for frontends come in. Instead of grouping every use case in a single application or delegating that responsibility to the UI, individual backends support the functionalities for specific UIs. For example, the product listing UI we discussed in Section [9.1](#) could be supported by a specific backend built for that use case. Other UIs like the order details UI and mobile interfaces would be supported each by its own backend, as illustrated in Figure [9-3](#).

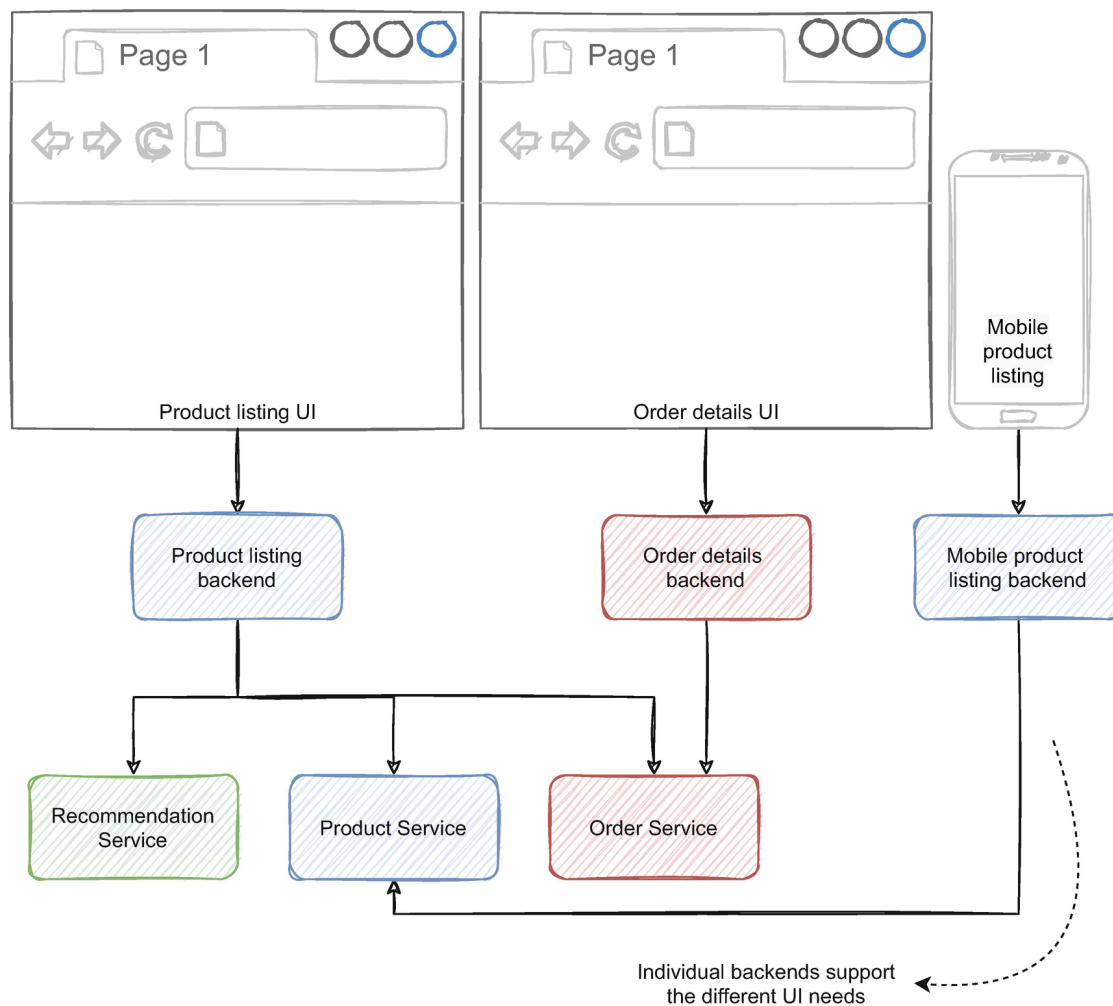


Figure 9-3 Each of the different UIs is supported by a backend built for that use case

In the example, we have a product listing and an order details UI hosted on a web browser and a product listing on a mobile device. Each of the three use cases is supported by its own backend, tailored for that purpose. There's usually concern about avoiding coupling the microservice platform design to the needs of the UI applications. For example, the order management platform functionalities should be agnostic of the order management application UI design. If another UI or even an integration with an external system decides to use the platform's functionalities, it should do so without further developments. Backends for frontends can fill that gap and adapt the requirements of the UI and platform without the concern of coupling to specific UI needs, since they are built for that purpose.

The example raises the question of how granular we should go on the design of the backend. For example, do we really need two product listing backends? Well, a reasonably good approach is to understand the use case and how the user interacts with the application. The web UI would

have information about recommendations and the order details; however, we might not display all of those sections in a mobile UI due to size and bandwidth constraints. The mobile UI would also likely show fewer products than a web UI would, and perhaps with a different interface (the way we go through the products could be different; on a web UI, we could use paging, while in a mobile, scrolling could be more appropriate). Those might be strong reasons to have an independent backend for each. The organization of the teams is also an important factor. If the same team develops both the web and mobile UI, it might be reasonable to have only one backend. As with microservices, having a backend accruing too much responsibility and several teams working on it can be good indicators to build a finer-grained one.

When there are separated teams for backend and frontend, BFFs can bring benefits to abstract the downstream platform functionalities and provide higher segregation of responsibilities. BFFs might not benefit simple applications where all functionality can be centralized in a single layer without dependencies, since they add complexity to the overall architecture. A reasonable approach is to use them when there is a complex logic to aggregate several services and a high diversity of use cases and frontend technologies.

BFFs still have limitations in larger data sets with more comprehensive search requirements. Saving state in BFFs, which is not unheard of with Redis and other technologies, might not be the best approach to a backend dedicated to serving a UI. BFFs and the patterns we discussed in Section [9.1](#) both rely on API composition, which can be hard to manage in certain use cases. Section [9.4](#) will detail these limitations and discuss an approach to solve them in event-driven architectures.

9.3 UI Decomposition Pattern in Microservice Architectures

As we discussed, one way to fetch information for a UI is to request data from the microservices. We can further optimize the requests by using an aggregating layer that fetches, joins, and shapes data to the UI needs. However, as our UI grows and requirements evolve, so do the responsibil-

ities of the UI. The UI application can grow and suffer from the same limitations of monoliths. This section will discuss how we can decompose a single UI application into smaller parts.

When UIs are responsible for a limited array of responsibilities and are simple and straightforward, we might benefit from using a single UI application. Like backend monoliths, a small team working in a single application might benefit from having a single application rather than dealing with the complexity of managing several smaller ones. However, as the application grows and accrues more responsibility, the developing teams also grow. Boundaries might become faded, releases become more complex and less common, and teams spend increasing amounts of time coordinating developments.

A pattern that arose to answer those difficulties is micro frontends. It relies on segregating the UI into smaller independently developed and deployed applications. Different teams own different micro frontends. It also provides the opportunity to have full-stack teams that own a smaller domain in the architecture and can implement both the UI and platform functionalities. Full-stack teams often can deliver value more consistently than having different teams for the backend and frontend and manage dependencies between the two. However, this approach requires the right company's context and ability to do so; but it might be more flexible delivery-wise than segregated platform and UI teams. Micro frontends can typically be implemented by application decomposition, page decomposition, and section decomposition, which are detailed by the following sections.

9.3.1 UI Application Decomposition Pattern

Application decomposition relates to segregating different purposes to different applications. It can be a valid approach when decomposing a single larger UI application into smaller ones. A comprehensive UI might aggregate several distinct responsibilities, which might benefit from having a segregated codebase, different team ownership and release cycle. It shares the same mindset of deconstructing a monolith into microservices; building smaller UI applications with a clear purpose for a set of domains can promote domain segregation. It can also be a catalyst for smaller

teams to build functionality from top to bottom, from the microservices to the UI, as we discussed before.

Clearly independent domains, use cases, and even user groups gathered in the same application can be signals that the UI application has too much responsibility and might benefit from segregation. For example, an eCommerce management application might benefit from having a segregated order fulfillment application and a customer management application, as illustrated in Figure 9-4.

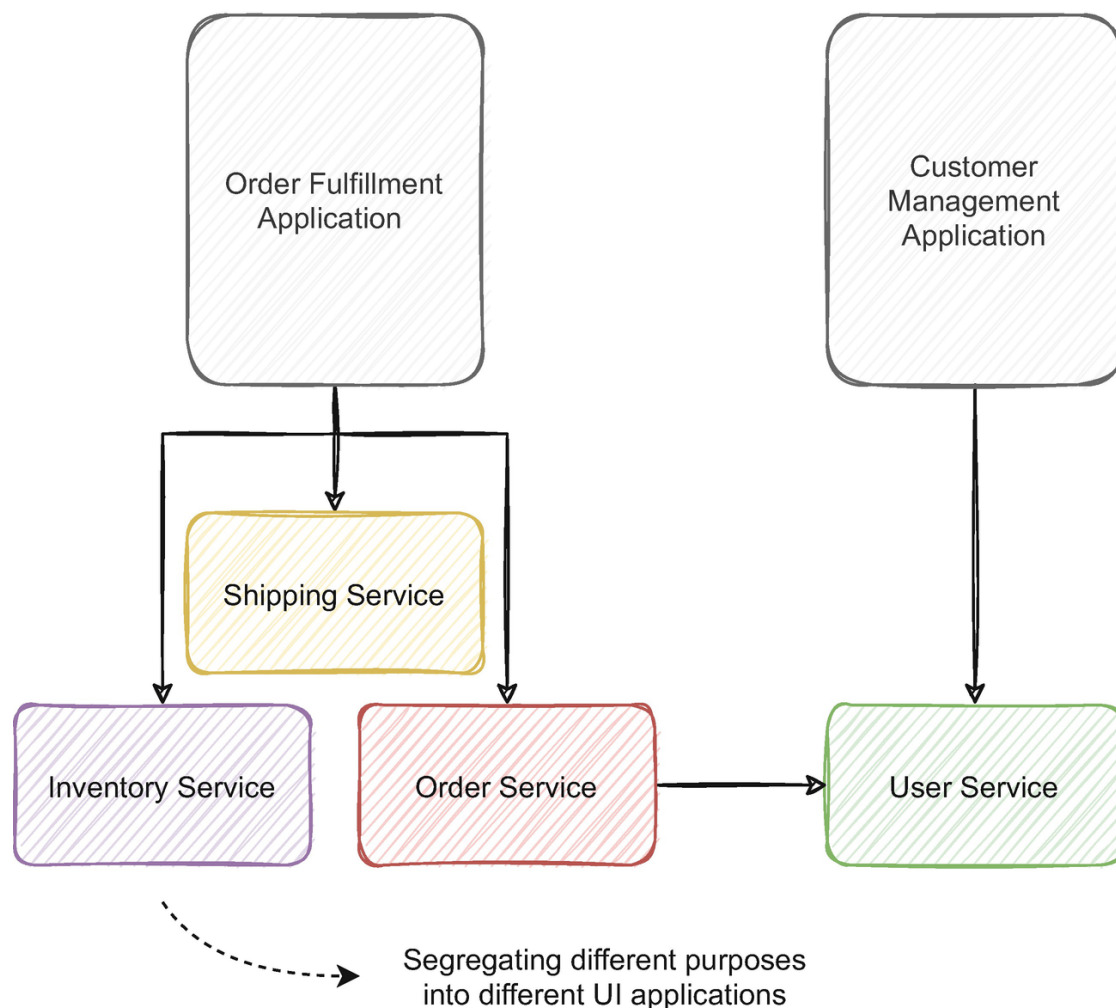


Figure 9-4 Using different applications for different purposes and business flows

In this example, the order fulfillment flow and the customer management application have two different purposes and business flows. Situations like this might benefit from dividing applications and building micro frontends for each use case. Ideally, each application would map to a single microservice; however, it can be hard to achieve as microservices get more fine-grained. The customer management application is a good choice for this approach; the team that develops the user service can also build the customer management application. The order fulfillment appli-

cation, since it comprises more significant responsibilities and concepts, might depend on several services; in those cases, we might benefit from finer-grained segregation like the following two patterns, page and section decomposition.

9.3.2 UI Page Decomposition Pattern

Page decomposition decomposes the application into a more fine-grained level by each page. We can look at an application by having different pages. For example, an order fulfillment application might have the functionality to manage stock, oversee the flow of the order, request order shipments, organize shipping providers, etc. Each functionality might be supported by a different set of pages, as illustrated in Figure 9-5.

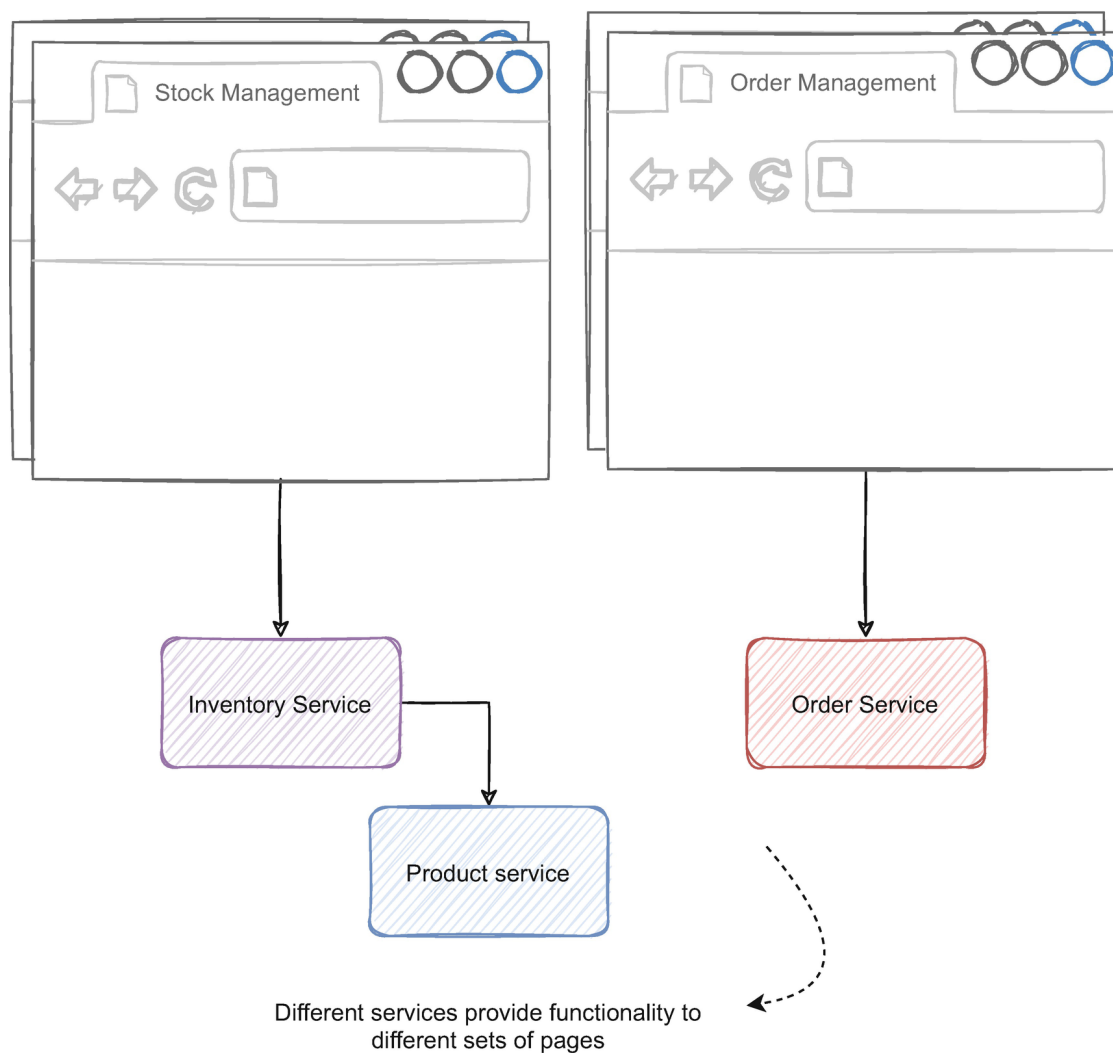


Figure 9-5 Different use cases are supported by different pages, which relate to segregated services

The functionality to manage stock is supported by the inventory and product service; the order management pages are supported by the order service. In this case, the inventory service might depend on the product

service to support the full functionality of the stock management pages, since the stock by itself might not mean much and might need to be enriched in the product information. However, this dependency doesn't need to be a direct synchronous dependency; the inventory service can build a custom view of the product data using the product event stream to support the UI.

The main advantage of this approach is not having the UI or an intermediate layer to request and aggregate data from several sources. The decomposition by page helps to have finer-grained segregation inside the same application, which might help to segregate responsibilities and boundaries. It also often translates more cleanly to the service platform. If we have pages for stock management, they will most likely rely on the inventory service, order fulfillment on the order service, shipping on the shipping service, and so on.

Page segregation often has a good translation to the Web; the navigation will be page-based which is aligned with the nature of the Web and has further synergy if different services are involved for each page. SPAs (single-page applications) became popular as a way to design applications to fit in a single page. SPAs have several advantages; for example, most resources (like scripts, CSS, HTML, etc.) are loaded one time which often makes the application faster. However, they can quickly become bloated for more complex applications; navigation often involves scripting. Multi-paged applications, on the other hand, often lead to simple developments; to navigate, we simply go to a different location on a separate page without the complex logic SPAs often have. Although they might not map so cleanly for mobile applications, they can be a good choice for more comprehensive applications on the Web.

9.3.3 UI Section Decomposition Pattern

There are use cases that might translate cleanly to a page and a microservice. A stock management page, for instance, might be a reasonable use case to apply it. However, there might be use cases where the entire page functionality might rely on several different services. This kind of aggregated view becomes an increasing concern as we build more fine-grained

services. Another similar, more fine-grained approach we can take is to segregate the UI by sections.

The example we discussed in Figure 9-1 had a UI that had a product listing, current order information, and recommendations for relevant products. Each section of the page has a reasonably segregated functionality which translates to different services, thus the need to aggregate results from various services on the UI or an intermediate layer. A different approach could be to compose the UI into different micro frontends for each section, which have their source in different services, as illustrated in Figure 9-6.

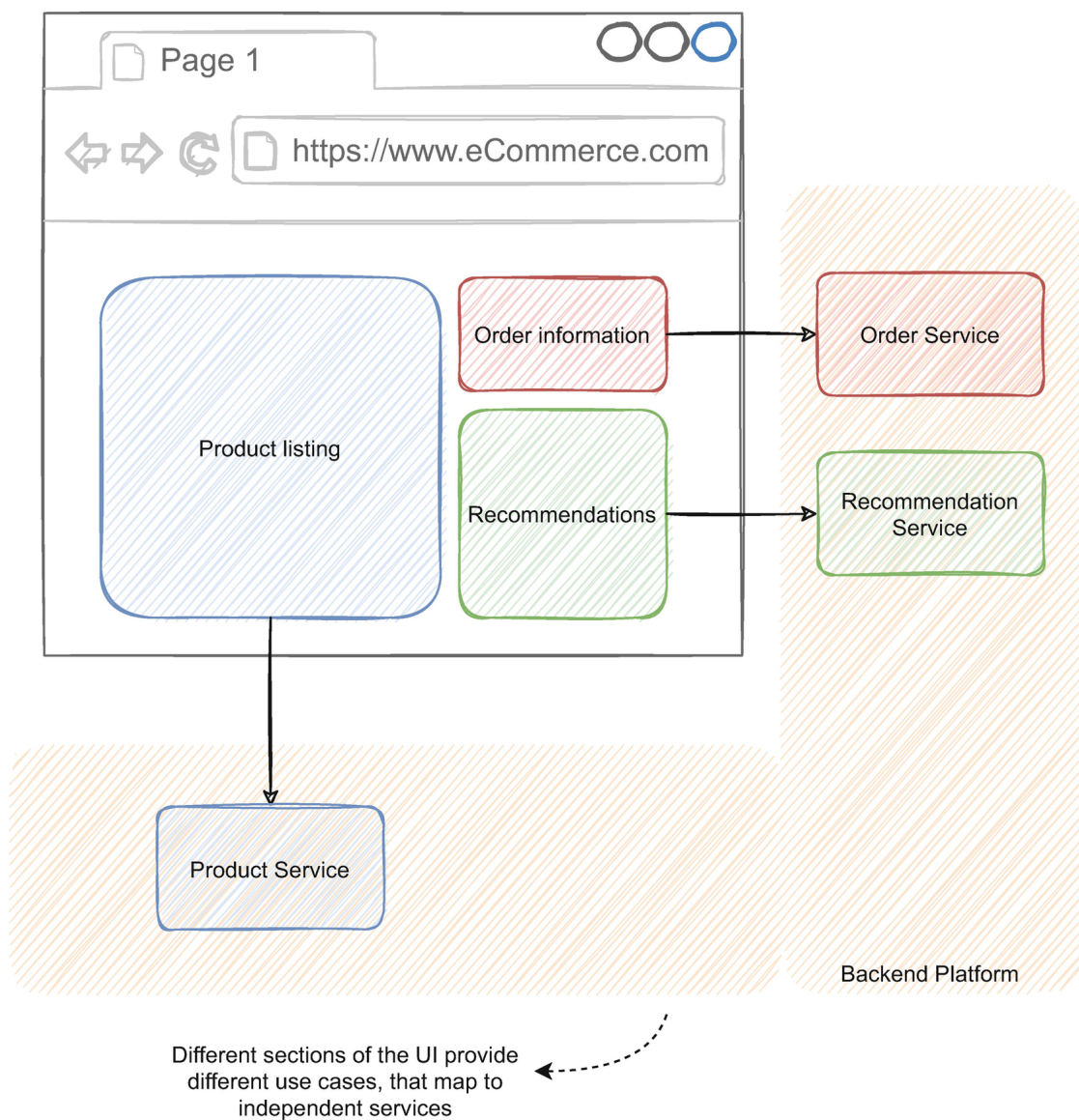


Figure 9-6 Relationship of different UI sections to independent services

Each section uses functionality provided by different services; for example, to list the product information, the product listing frontend can rely solely on the product service. We might still need to do three separate requests to provide the UI's full functionality. Still, having segregated

sections for each use case, we can adapt to show only a subset of the sections on UIs more sensible to bandwidth limitations (like mobile), depending on the page's use case.

An important consideration is the ability to build independent sections backed by independent micro frontends. Each section can be individually deployed, meaning we can implement new functionality on the recommendation section and deploy it without changing the product listing and order information section. There's still the need of a UI to stitch the sections together; however, the main functionality is supported by the section's frontend. This flexibility to segregate into different frontends facilitates the collaboration of different teams to the same UI, which could be hard to do with an application or page decomposition.

It also promotes the build of full-stack teams; the product listing and the product service could be developed by the same team and deliver value independently. The ability to have a team with autonomy to deliver new features from top to bottom often facilitates delivery and makes delivering value faster and more straightforward for the business.

When we can segregate use cases into different pages, a page decomposition can be more beneficial. With page decomposition, we can avoid the complexity of composing and supporting the communication of different sections. Section decomposition can be a valuable approach in complex and more comprehensive single page UIs that provide a wide range of features. It can be a good approach to segregate responsibilities and provide a scalable way for different teams to contribute to the same UI.

9.4 The Limitations of API Composition

Section [9.1](#) discussed how the UI or an aggregating layer fetched information from the microservice platform using synchronous requests. Aggregating data from several microservices can be a valid approach when the data is small and is centralized in a small number of services. For example, to build a UI to provide the details of an order, we would likely need a couple of requests to the order and product service. On the other hand,

large sets of data and requirements to aggregate data from different sources can be troublesome to deal with. Fetching and aggregating information from different sources is usually referred to as the API composition pattern. As you might recall, we briefly discuss API composition in Chapter [4](#); this section will further detail and work through a use case detailing the limitations of this approach and when it is most helpful.

In the example we discussed in the beginning of the chapter in Figure [9-1](#), different areas of the UI use different information. In that situation, it might be manageable since each section doesn't have a strong relation to the other ones. The product listing doesn't have a strong relationship with the recommendation and order section, for example. But aggregating data from different sources can be extensively limiting when a single component has to merge the information and provide it in the same section. For example, the listing page would likely have to provide a more aggregated view of the product catalog with information relevant from different domains. An example of a UI with this functionality is illustrated in Figure [9-7](#).

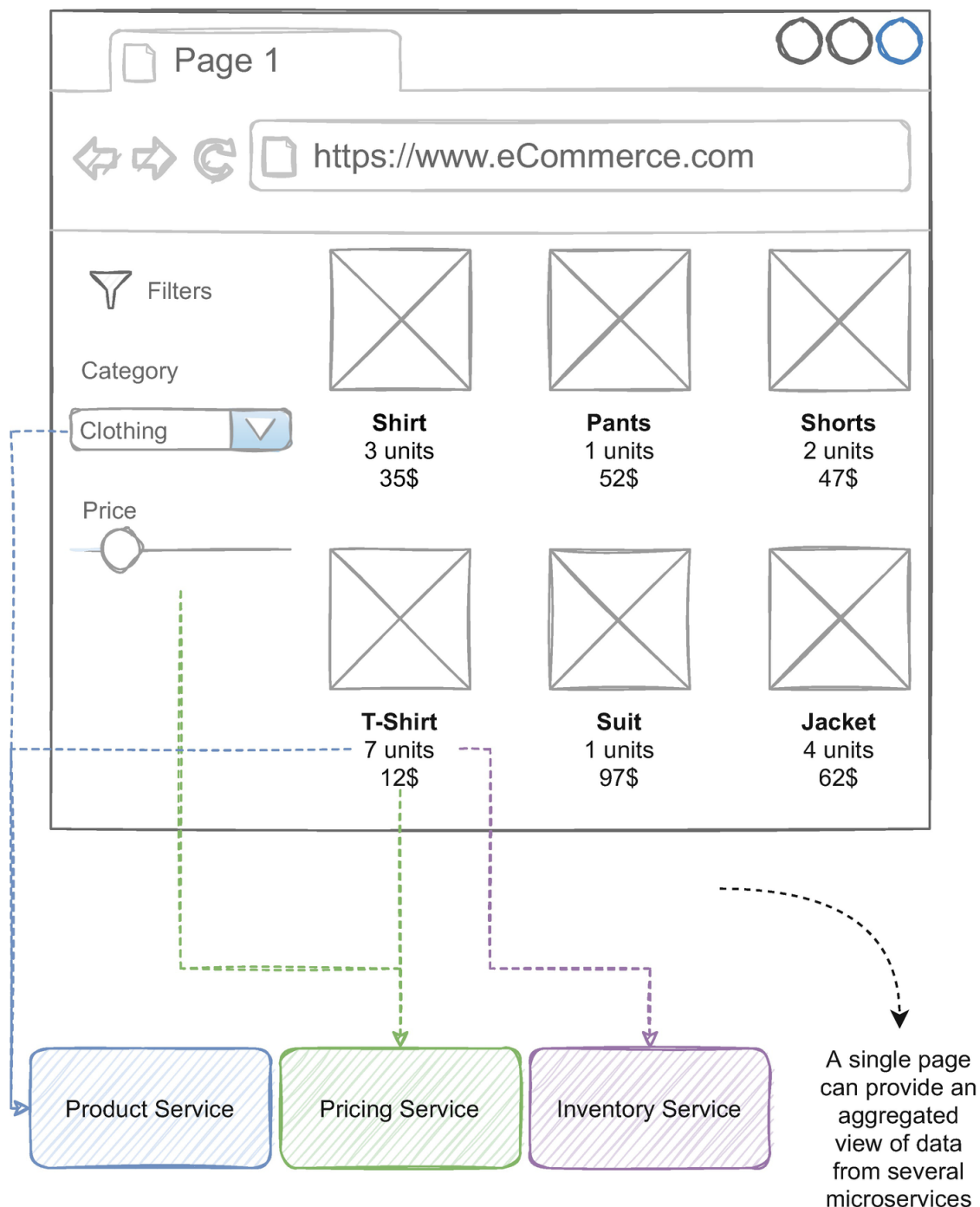


Figure 9-7 Example of a product listing page providing information from different sources

The product listing page provides information about the product description, current available stock, and price of the item for the user's country. It also allows the user to filter products by category and define a price range. These kinds of requirements are fairly simple use cases; in a real situation, a more comprehensive list of filters would likely be available along with sorting capabilities.

Using API composition, the application would request the information from each individual service, which in simple use cases might be straightforward. However, when there is a need to do more comprehensive oper-

ations on the data across several services, it becomes extensively challenging or even prohibiting in data-intensive use cases. In this example, the listing page displays six products; it might be affordable to request the six products from the product service and the price and stock quantity from the pricing and inventory service; it would likely only take three requests. What if we apply a category filter? It would be similar; we apply the filter in the request to the product service and fetch the results information from the other services. However, what if we filter by category and use a price range, for example, all clothing items with a price lower than 50\$? We would need to apply the category filter to the product service and the price filter to the pricing service. If we request the six clothing products from the product service and then fetch the price, only a subset of them might be lower than 50\$. We would need to make a series of subsequent requests until we find a set of products that oblige all criteria; the number of needed requests can quickly become infeasible. Imagine now we only want to display products with stock; we would need to fetch results from the three services that oblige to all filters. If we add sorting, it gets even trickier.

Eventually, more comprehensive searches are required by the business. Using API composition in these use cases with considerable quantities of data will require overwhelming requests by the UI or the aggregating layer, often leading to poor performance. Perhaps we can afford making a few requests to fetch information, but when there are hundreds of thousands of entities in each service, it becomes unacceptable to fetch all data from the downstream services and join it in memory. It also would have a considerable impact on the resources of the component aggregating the responses. Imagine if the products that fulfill the results aren't on the first pages, the component would have to fetch page by page, keep the results, join them with the responses of the other services (which might also be at the end of the result list), and select the ones that comply with the filters. If hundreds of users made the same request, it would definitely impact the component's memory and CPU. Elasticsearch actually works in a similar way¹ and warns about the impacts of deep paging, which might bring down nodes due to lack of memory. By applying this strategy to complex requirements, we would likely suffer from similar limitations.

Some requirements might need an aggregated view of the data to support more comprehensive ways to look at the information. Event-driven architectures are designed in a way that supports data sharing efficiently. We could continuously build an optimized view of the data by handling the different event streams, as illustrated in Figure [9-8](#).

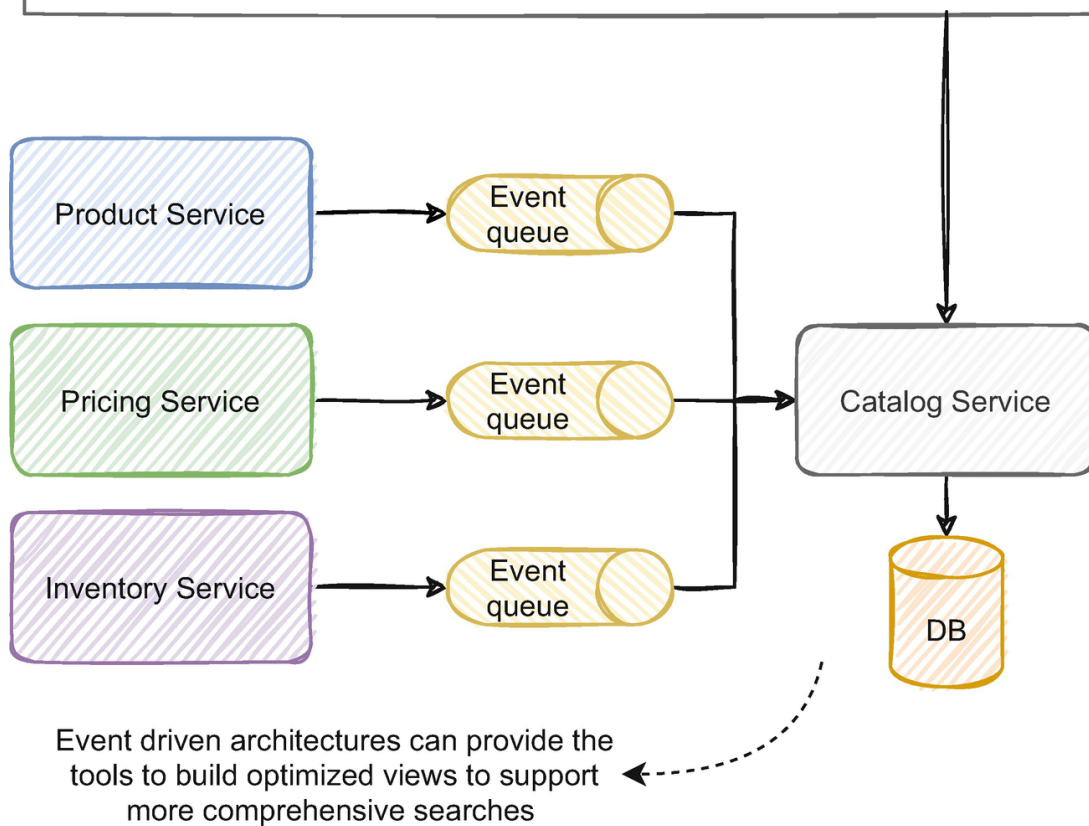
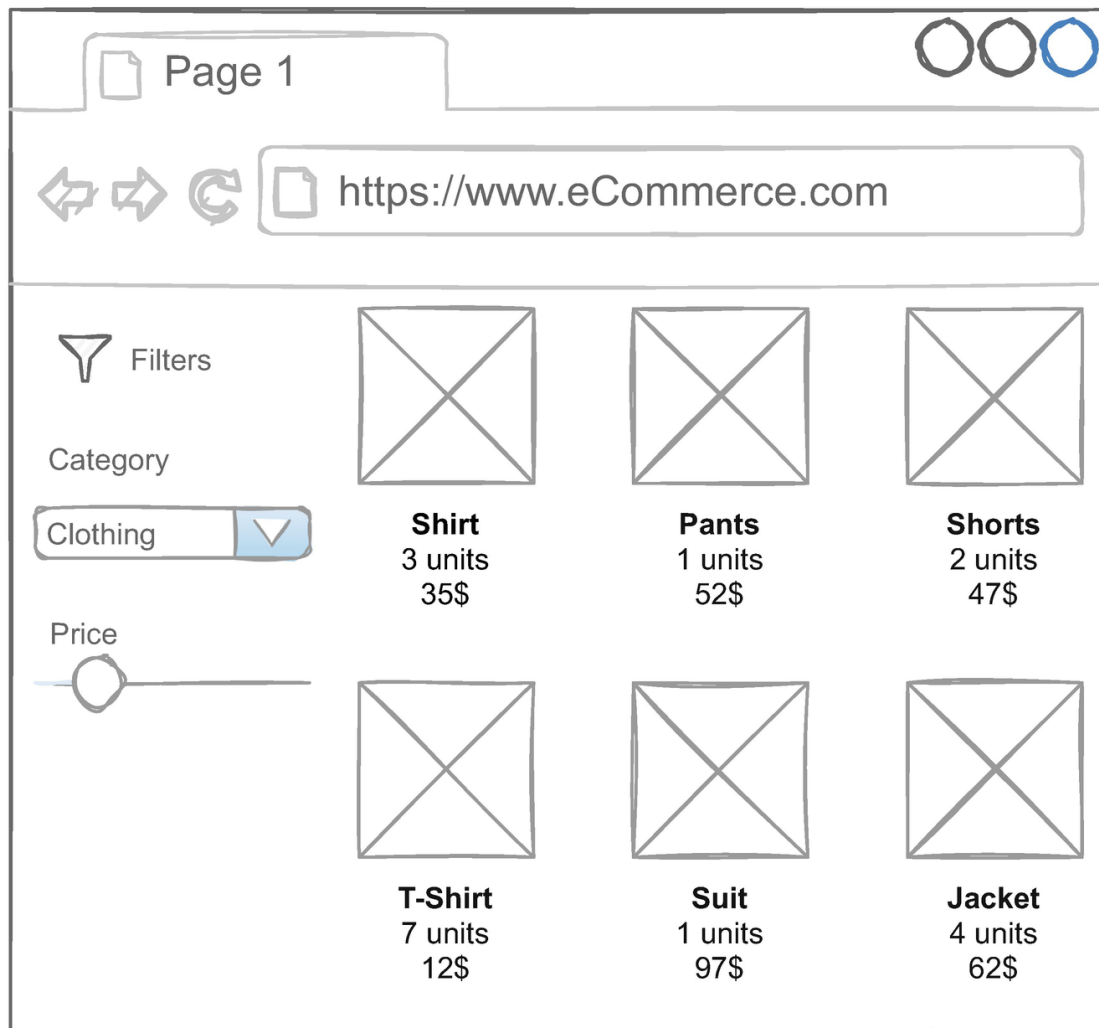


Figure 9-8 We could build a denormalized view of the data using the event streams of each service

This approach is very similar to the CQRS pattern. Each service acts as the write model and owns the rules and logic of each domain, while the

catalog service simply builds a denormalized view of the information of each service. This view is optimized to the searching requirements and is able to support a more comprehensive view of the data. An essential consideration in this approach is guaranteeing the domain logic exclusively relies on the respective service; for example, the pricing domain logic can't exist in the catalog service, solely in the pricing service. The catalog service should only handle the information relevant to the use case it is supporting. For example, the discount rate might be relevant information, but if there isn't any use case to support it, then the catalog service should ignore it (using an anti-corruption layer, for example).

An important pitfall is to transform the catalog service into a monolith in its own right. If more fine-grained services start to appear, we might feel the need to increasingly add functionality and different kinds of searches to the catalog service. It really depends on the use case, but following the same approach of BFFs, using a model per use case or user experience might be a sensible approach. If the service has too much responsibility, we should consider splitting it.

Another important concern is the ownership of the service. Ideally, the same team would be able develop the UI and own this component to allow the implementation of new functionalities without dependencies on other teams. Developing the backend functionalities based on the event streams and the UI, features tend to be delivered faster. However, depending on the organization might be hard to find the skills to develop both components.

API composition is still valid; for example, a UI that displays the user's order details might only need a couple of requests to the downstream services to retrieve the required information, and we would hardly benefit from having a dedicated service to build a denormalized view. However, it has limitations; if you find API composition has enough performance to support the use case, it usually is the easiest and most straightforward solution. In use cases that require a more comprehensive view and filtering, building a dedicated model might be a reasonable approach to guarantee acceptable performance.

9.5 Task-Based UIs

Until now, we discussed the difficulties we can find when adapting the backend functionalities to the frontend UI requirements and the patterns we can use to overcome those challenges. However, we haven't detailed how the UI could actually look like. Although UI design and the concepts of UI user experience are beyond the concepts of this book, event-driven architectures introduce different challenges in the way users interact with the application and the overall system. A reasonable understanding of those challenges and a deliberate approach to UI design can substantially smooth the interaction users have with the platform. This section will discuss how task-based UIs can help ease that interaction and provide a smooth transition to an event-driven, domain-oriented architecture.

Traditional UIs are typically CRUD-based. We open a UI, the application fetches the current state, we change the data in some forms and submit the changes, and the application persists the new state in the database. For example, imagine we had an application to create products for an eCommerce platform, we could have a UI with several forms to submit the product's information, for example, the brand, category, price, stock units, etc. Once we filled in the required fields, we could submit the product data, as illustrated in Figure [9-9](#).

Page 1

https://www.eCommerce.com

Display card

Friendly name:

Embroidered cotton sweatshirt

Product information

Description: Elevating your everyday look just got easier. This sweatshirt is a casual piece you can always count on. Look and feel your best with the help of this brand.

Brand: ABrand

Category: Sweatshirts

Status: Online

Vendor details

Stock: 3

SKU: QPO-ACD-G463

Price: 18\$

Discount: 10%

Submit

Example of a CRUD UI for editing products

Figure 9-9 Example of a CRUD UI to submit a product edition

The product information is submitted in a single interaction; it's limited to data entry rather than a business process. This kind of approach is simple, straightforward, and easy to understand and benefits simple use cases where the application doesn't need to model a complex domain. The process is limited to transferring and manipulating a DTO (data transfer object); the service fetches the data, maps to a DTO, and passes it to the UI; the UI changes it and sends it back to the service to be saved.

As you might recall, when we discussed DDD, commands and events should reflect the user's intent; the very business domain value is imbued in their design. In CRUD style UI, the user's intent is often lost or has very limited meaning; the interaction is resumed in manipulating the current state.

Following this approach with more complex domains and more intricate business processes often leads to the application failing to capture the real domain workflow. You probably experienced this firsthand a time or another when the people operating the software had to edit information on a series of applications or UIs. The workflow of the operations is written down on a piece of paper or in someone's mind (like go to UI X and edit something, go to UI Y and change something else). Is that an intricacy of a given system/business, or just the failure of the system to model what the user needs to do? Certainly, this kind of design is valuable in simpler systems; more complex domains and processes might benefit from different approaches. The example in Figure 9-10 also highlights an additional challenge when mapping to a microservice architecture; there are different sets of information belonging to different domains.

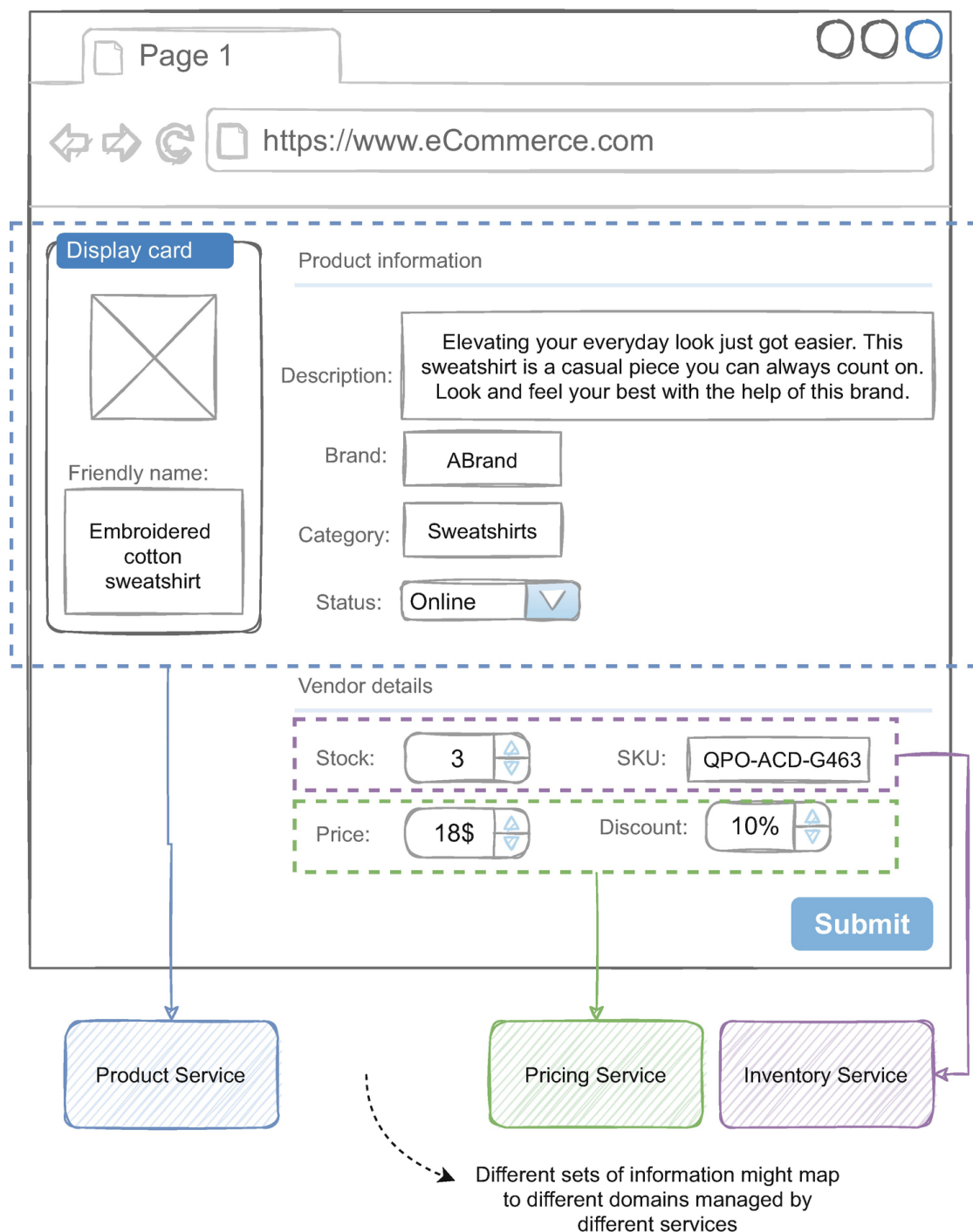


Figure 9-10 CRUD UI coupling together different domains

Having a UI aggregating different domains together isn't necessarily bad, and it often is a requirement by the business. But changing information in different domains usually implies different actions and intentions. It also begs the question of how to guarantee the consistency of a single update. For example, by pressing the Submit button, the inventory service might save the stock information successfully, while the product service might fail to submit the product information due to an error. What should we show when one service fails but the others succeed? Displaying an error message saying "Submission failed for part of the information" might be an approach but certainly is a dubious user experience.

Task-based UIs are typically built to help the user advance in a journey and guide him to accomplish a process. Each change also maps more cleanly to the user's intent, making it easier to map to commands for each domain. For example, we could model the UI to translate each user's action by segregating the different tasks, as illustrated in Figure [9-11](#).

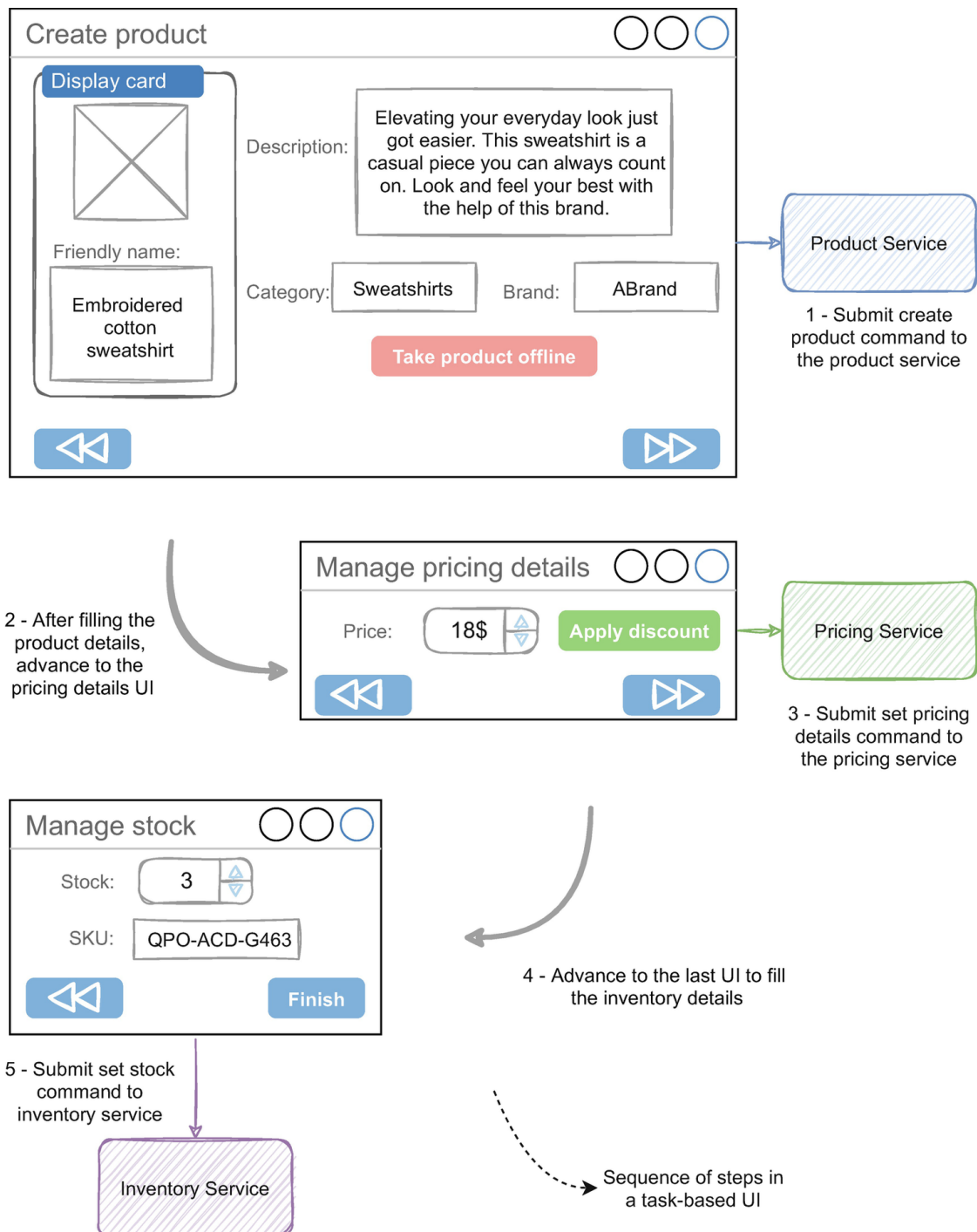


Figure 9-11 Example of task-based UI for product creation

Each context has a different screen and maps to a respective service and domain. For example, the screen to manage the product's price is dedicated to that task, reflecting the user's intent more clearly. This design also helps to translate the user actions to commands. A change product price command semantic is more meaningful and has more domain value than an edit product command.

Task-based UIs usually map more cleanly and have higher synergy with distributed architectures due to building commands for each segre-

gated task. Each task has a separate command, which most likely will translate to an event with similar meaning, paving the way for more meaningful and domain-rich event streams. Event streams that capture the user's intent tend to be more valuable and more relevant for future requirements.

Typically, simple domains with straightforward processes don't benefit much from this kind of approach. If the user's operations are CRUD in nature, then a CRUD approach is obviously more beneficial. Event-driven architectures typically model complex domains; however, don't fall into the trap of applying this kind of design everywhere. In complex architectures, there are several domains; some are intricate enough to benefit from this approach, while others might hardly benefit from it.

Arguably, the most useful benefit we can extract from task-based UIs, even when we end up not using them, is the focus on why and how the user is trying to achieve its goal. The usability and the nature of the business process are the cornerstones of the design.

9.6 Event-Driven APIs

The asynchronous nature of event-driven APIs often transforms an otherwise synchronous operation into an asynchronous one. For example, the order fulfillment process in a single-process application like a monolith could be a synchronous process for the order submissions that have all the conditions to be fulfilled. However, in an event-driven architecture, the order fulfillment is likely the choreography of several event-driven services. Asynchronous operations might be more common throughout the architecture. To ensure a friendly web experience, we might shift from synchronous CRUD UIs, where feedback was immediate, to UIs that show the progress of the request as it advances in the workflow, displaying a status or alerts to the user.

With the advent of reactive UIs, event-driven frontend implementations are becoming increasingly popular. They have high synergy with the asynchronous approach; as the state advances, asynchronous updates can be delivered to the user seamlessly. Although older protocols like

XMPP (Extensible Messaging and Presence Protocol) are still popular and traditionally used to provide this functionality, more recent and light-weight alternatives are available. WebSockets ² have become quite popular recently as a solution for two-way communication and bi-directional traffic. Server-Sent Events ³ is also a valuable alternative approach, although focused on pushing new data to the client in one-way communication. WebHooks ⁴ are also an interesting approach to implement one-way notifications to clients using custom callbacks. All three approaches are valid choices for asynchronous real-time communication, much like event-driven microservice architectures.

It's peculiar how both backend and frontend embrace event-driven mindsets (although with very distinct use cases, tools, and at different scales), but the communication between them often uses synchronous HTTP API calls. In fact, for a long time, there was a scarce number of options to provide seamless backend asynchronous interfaces for UI applications to use. Microservice message brokers like Kafka or RabbitMQ might not be suitable to deliver asynchronous events to UI applications due to challenges with firewalls, lack of granular role or resource-based access control, and lack of clear standards and straightforward ways to access asynchronous events.

In fact, traditional synchronous APIs like REST are a common functionality in most platforms in no small measure due to clear, well-established standards. Standards like OpenAPI ⁵ simplified the development and integration of different systems without the need for complex coordination. AsyncAPI ⁶ is increasingly filling that gap in asynchronous APIs. It is an open source project based on OpenAPI that seeks to provide a specification for asynchronous implementations with a message-driven foundation. It supports several messaging protocols and provides tools to define the event schema and semantics to connect, subscribe, publish, and interact with the API. It also supports a wide range of tooling, similar to OpenAPI, like code generation, API discovery, and event management. It can be a valuable approach for documenting and designing your asynchronous APIs.

9.6.1 Event-Driven Combined with WebSockets

Let's work through an example of how an asynchronous functionality could interact with the UI using WebSockets . Let's say we have a UI to create products and the business only considers the product created once it is available on the product catalog to be managed. The product catalog is updated asynchronously, so it would be useful for the user to know when the product is available on the catalog. Figure [9-12](#) illustrates this example.

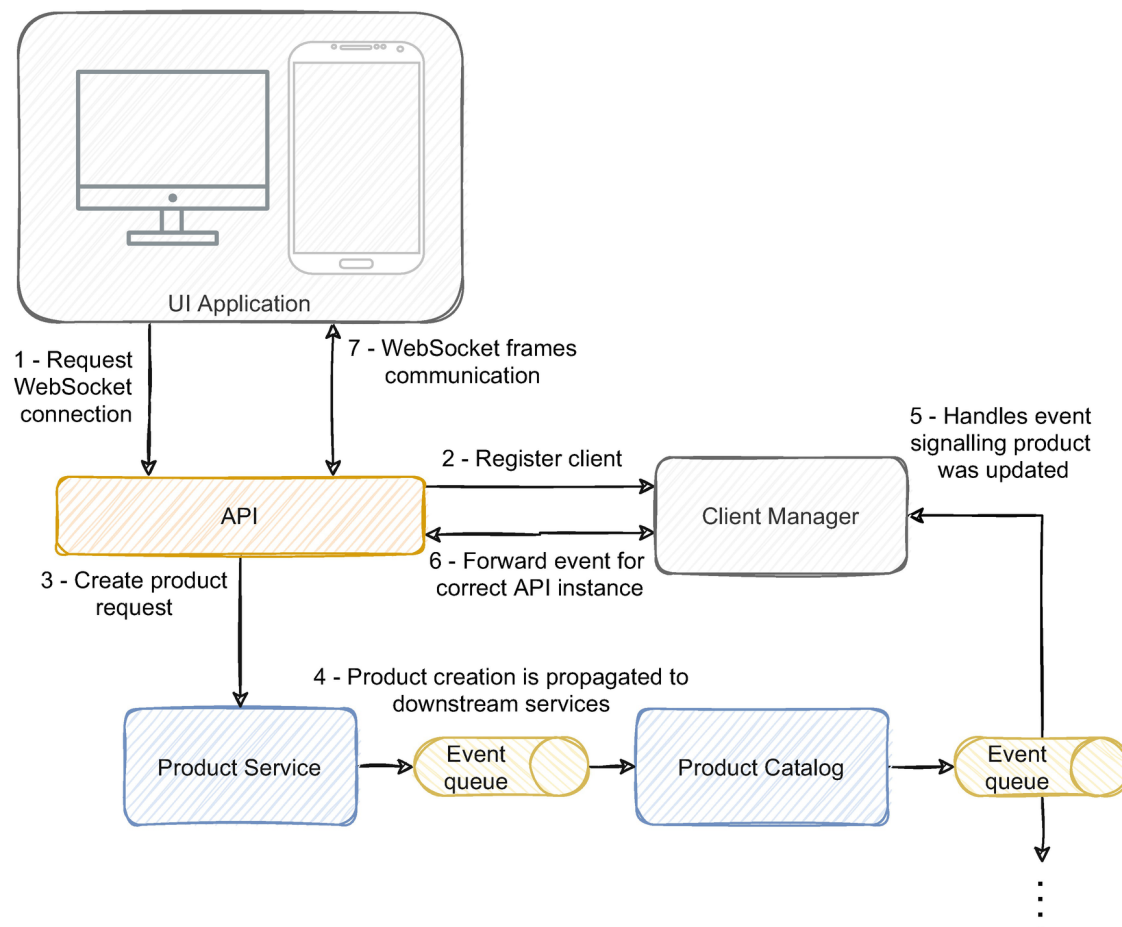


Figure 9-12 Asynchronous product creation using WebSockets

The user creates the product through the UI application, which triggers a request to the API component and the product service. The product service creates the product and publishes an event signaling that change, which the product catalog service uses to register that product in the catalog. The product catalog, which likely handles different streams to enrich a denormalized model of the products (similar to what we discussed in Figure [9-8](#)), also publishes an event signaling the product creation.

The UI application first registers itself on the API and requests a WebSocket connection. The client manager is responsible for keeping track of the registered clients and handling the event signaling the product cre-

ation in the catalog. Each time a product is created, the client manager handles the event and informs the UI of successful product creation.

Why do we need the client manager component? Can't we just handle the events from the API? The challenge of delivering events to the front-end applications is delivering the correct events to the right clients. All components can (and likely will) be horizontally scaled, meaning there will be several instances of each component. If there are several product catalog services, UI applications, and API instances, how can we guarantee the events will be delivered to the correct client that created the product?

A possible solution could be for each of the API instances to handle all events and filter those that correspond to the clients connected to that instance. It can be an option for low-scale use cases; however, it quickly becomes impracticable for large event streams. The client manager can help manage all existing clients and deliver each event to the corresponding API instance, which has a WebSocket connection to the client.

9.6.2 Event-Driven Combined with Server-Sent Events

In use cases, we don't need a bi-directional communication channel; Server-Sent Events can be a valid alternative. Server-Sent Events is a subscribe-only protocol that provides a way to subscribe to an event stream and is a lightweight approach to deliver asynchronous data to the front-end applications.

The approach to the use case we discussed in the previous section with Server-Sent Events remains largely unaltered. Instead of using a two-way connection to the backend, the UI application opens a persistent connection to the API with the `EventSource`⁷ object. The client manager component forwards the event stream to the API. Figure [9-13](#) illustrates the same use case we discussed before.

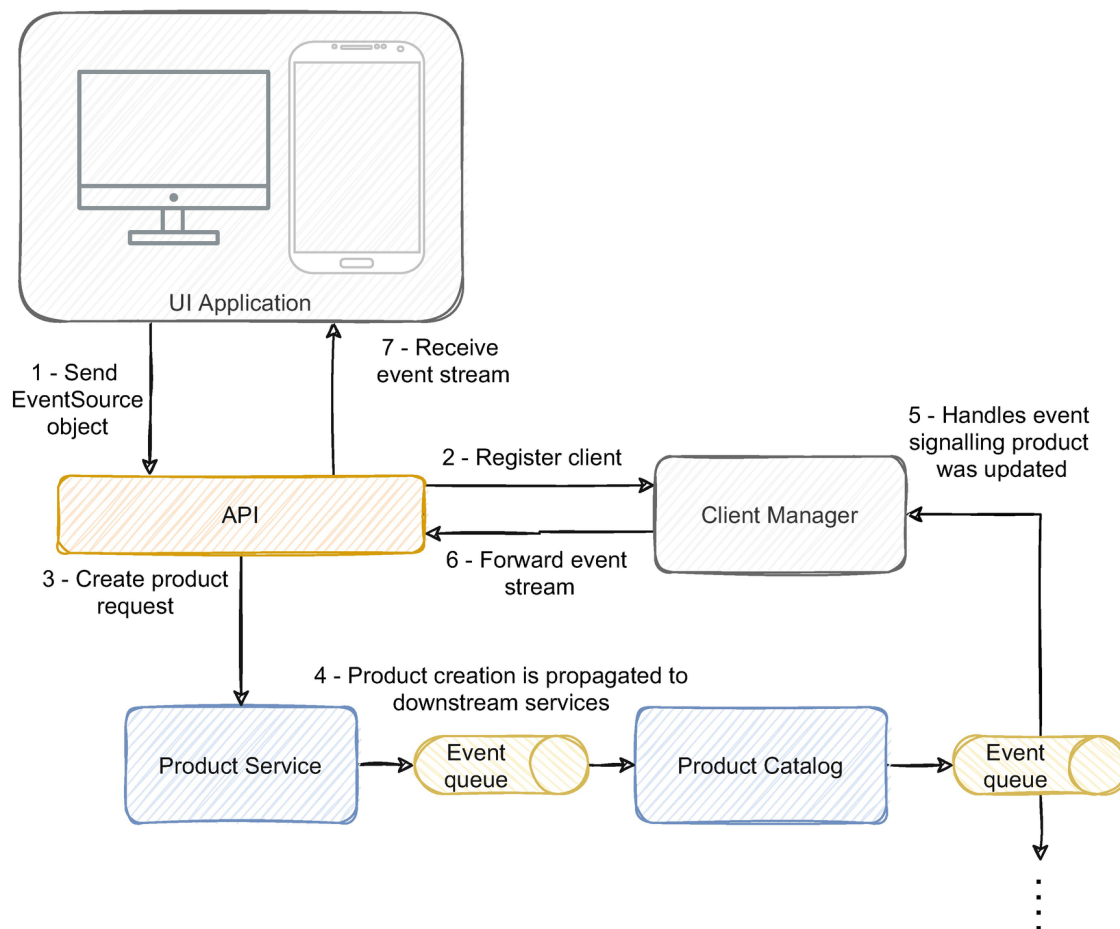


Figure 9-13 Asynchronous product creation using Server-Sent Events

Instead of having a bi-directional connection, the delivery of the event stream is one-way. Server-Sent Events also supports straightforward error handling, generating error events that can be more sustainably dealt with in the frontend application. One main difference from WebSockets is they use HTTP requests to maintain a persistent connection (also getting multiplexing over HTTP/2) instead of a dedicated protocol.

WebSockets tend to be more heavyweight in mobile use cases; since they establish a two-way connection, they require a full-duplex antenna, which in most use cases impacts the bandwidth and power consumption. Server-Sent Events implements unidirectional traffic through HTTP which, compared with WebSockets, tends to reduce the total data usage and battery power.⁸

9.6.3 Event-Driven Combined with WebHooks

Another alternative to implementing one-way asynchronous communication to the frontend applications is through WebHooks. WebHooks work

in a reverse flow that we are used to; the subscriber registers a callback that the server will call to deliver the event stream (they are often dubbed as reverse APIs). Similar to Server-Sent Events, the communication occurs through HTTP and doesn't apply any specialized protocol.

The main difference from the other alternatives is the callback mechanism. Let's work through the same example we discussed in the previous section using WebHooks. The product creation flows through the product and product catalog service, and we deliver the corresponding event to the frontend application, as illustrated in Figure 9-14.

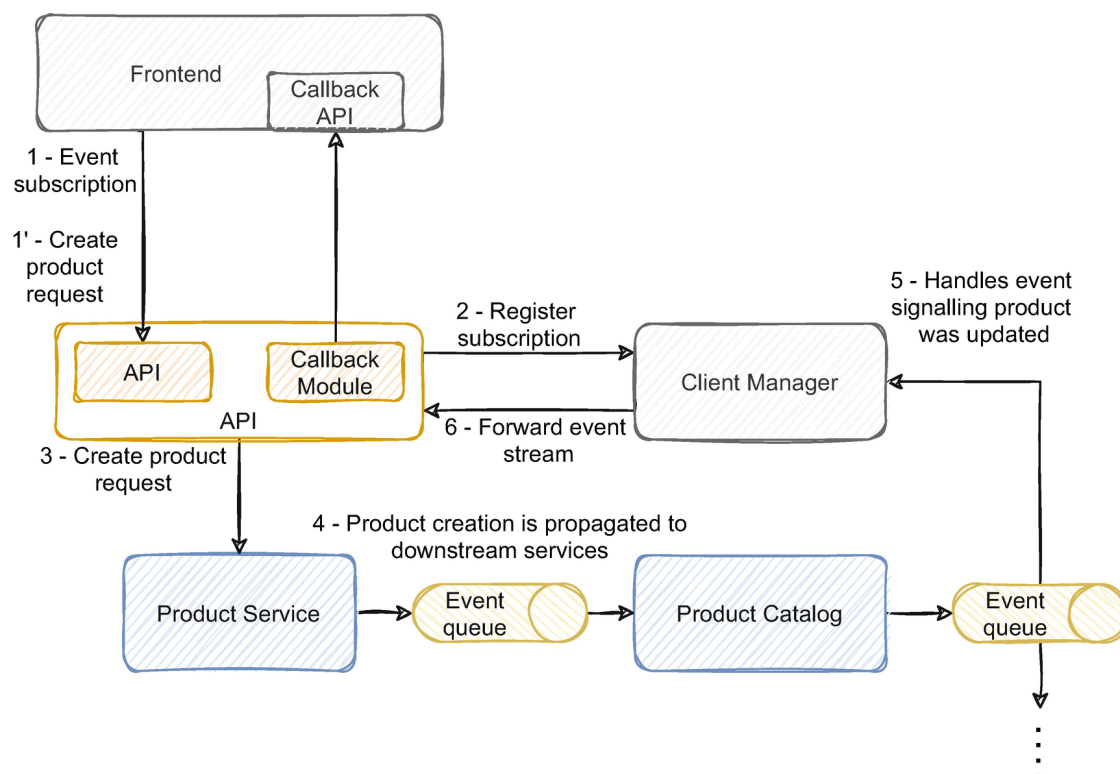


Figure 9-14 Asynchronous product creation using WebHooks

The frontend application subscribes to the event stream with an HTTP request to the API and registers a callback URL, which the backend will use to deliver the event stream. The product creation flows through the services, and the product catalog service will eventually generate an event signaling the change. The client manager will handle that event and deliver it to the callback module responsible for delivering the stream to the frontend application. To do so, the module will typically make a POST request to the callback API providing the new events.

The limitation of WebHooks resides in relying on an HTTP callback endpoint to deliver the events. Hosting a publicly exposed HTTP endpoint

in user-facing applications like mobile apps or a browser is often impractical or even infeasible due to implementation and security concerns. Typically, WebHooks are applied to frontends or BFFs which are servers and can sustainably implement this feature. However, they are an interesting option when we need one-way communication for push notifications or event delivery.

9.7 Summary

- An aggregating layer can help optimize the bandwidth and response time when we need to aggregate data from different services. It's typically best applied to simple applications and contained use cases. We should avoid the pitfall of adding too much responsibility to the aggregating layer; if code has a place to gravitate to, it typically will.
- BFFs can abstract the downstream platform functionalities and provide higher segregation of responsibilities, thus avoiding adding too much responsibility to a single aggregating layer. They also enable a close optimization of the UI needs without the worry of coupling platform features to specific UI use cases.
- We can decompose a UI application into smaller components to best adapt to the underlying microservice platform. Decomposition can be done at the application, page, and section level; we can adapt each pattern to the most adequate use case.
- API composition is a useful and straightforward solution for obtaining the data to display in a small number of requests. However, in use cases that require comprehensive filtering and ordering requirements, building a dedicated model might be a reasonable approach to guarantee acceptable performance.
- Task-based UIs usually have higher synergy with distributed architectures due to building commands for each segregated task. They also are typically built to help the user advance in a journey and guide him to accomplish a process. Each change also maps more cleanly to the user's intent, making it easier to map to commands for each domain.
- Event-driven APIs can be a valuable alternative to traditional synchronous APIs in event-driven architectures. We can use them to asynchronously inform the user of changes in the ecosystems in a sustainable manner.

- To implement event-driven APIs, we can use WebSockets, Server-Sent Events, and WebHooks. WebSockets provide a way to implement two-way communication between the UI and the backend. Server-Sent Events and WebHooks are good options for one-way event delivery and push notifications.

Footnotes

- [1](https://www.elastic.co/guide/en/elasticsearch/reference/current/paginate-search-results.htm) Article in Elasticsearch documentation, “Paginate search results,” www.elastic.co/guide/en/elasticsearch/reference/current/paginate-search-results.htm
- [2](https://datatracker.ietf.org/doc/html/rfc6455) Protocol definition in “The WebSocket Protocol,” December 2011, <https://datatracker.ietf.org/doc/html/rfc6455>
- [3](https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events) More information in “Server-send events,” https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
- [4](https://en.wikipedia.org/wiki/Webhook) Wikipedia page in “WebHook,” <https://en.wikipedia.org/wiki/Webhook>
- [5](https://spec.openapis.org/oas/v3.1.0) Specification in “OpenAPI Specification,” February 15, 2021, <https://spec.openapis.org/oas/v3.1.0>
- [6](https://www.asyncapi.com/docs/specifications/v2.0.0) Specification in “AsyncAPI Specification,” www.asyncapi.com/docs/specifications/v2.0.0
- [7](https://developer.mozilla.org/en-US/docs/Web/API/EventSource) More information in <https://developer.mozilla.org/en-US/docs/Web/API/EventSource>
- [8](https://www.smashingmagazine.com/2018/02/sse-websockets-data-flow-http2/) Full article in Martin Chaov, “Using SSE Instead Of WebSockets For Unidirectional Data Flow Over HTTP/2,” February 12, 2018, www.smashingmagazine.com/2018/02/sse-websockets-data-flow-http2/

